This is a massive undertaking. To go "full stack" with the depth of Fiverr, you aren't just building a website; you're building a complex ecosystem with distinct, secure workflows for two primary users, plus administration.
Here is the complete, integrated **Product Requirements Document (PRD)** for **FreelanceX**. It defines the architecture, database schemas, and the specific UI flow for every single page, using Fiverr's role-based separation as the functional blueprint.

PRODUCT REQUIREMENTS DOCUMENT (PRD)

Project Name: FreelanceX

Full-Stack Architecture, Implementation Blueprint & Developer Specification

1. Executive Summary & Core Objective

FreelanceX is a secure, high-performance, full-stack freelance marketplace web application engineered to bridge the gap between employers and skilled freelancers. It features milestone-based escrow payments powered by **Paystack**, real-time peer-to-peer communication via **Socket.io**, automated transactional email notifications via **Brevo (SMTP)**, a robust dispute resolution reporting system, and a comprehensive admin moderation panel.

- **Tech Stack:**
  - **Frontend & Routing:** Next.js 14+ (App Router), TypeScript, Tailwind CSS.
  - **Backend & API:** Node.js, Express.js, Apollo Server (GraphQL API route), Socket.io.
  - **Database & ODM:** MongoDB with Mongoose.
  - **Payments:** Paystack API (Checkout initialization, webhook verification, automated bank transfers).
  - **Email Service:** Brevo SMTP (via Nodemailer) for transactional email notifications.
  - **Design System:** Clean white backgrounds (#FFFFFF / #F8FAFC), deep charcoal typography (#0F172A), muted slate body text (#475569), and professional indigo interactive elements (#4F46E5).

2. Environment Variables Specification (.env)

Create a .env file in the root directory of your project. This includes your **Brevo SMTP** configuration credentials:

Code snippet

```
# ==========================================
# FREELANCEX ENVIRONMENT CONFIGURATION
```

```
# ==========================================

# Application Port & Host
PORT=5000
NODE_ENV=development
NEXT_PUBLIC_SITE_URL=http://localhost:3000
NEXT_PUBLIC_BACKEND_URL=http://localhost:5000

# Database Connection
MONGO_URI=mongodb+srv://<username>:<password>@cluster.mongodb.net/freelancex?
retryWrites=true&w=majority

# Authentication & Security
JWT_SECRET=super_secret_jwt_key_change_in_production_12345
JWT_EXPIRES_IN=7d

# Paystack Payment Gateway Keys
PAYSTACK_SECRET_KEY=sk_test_xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx
PAYSTACK_PUBLIC_KEY=pk_test_xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx

# Transactional Email Configuration (Brevo SMTP)
EMAIL_HOST=smtp-relay.brevo.com
EMAIL_PORT=587
EMAIL_USER=your_brevo_login_or_smtp_email@domain.com
EMAIL_PASS=your_brevo_smtp_master_password_or_key
EMAIL_FROM=noreply@freelancex.com
```

3. Directory Structure & File Architecture

Plaintext

```
freelancex/
├── .env                    # Environment configuration file
├── package.json
├── server.js               # Express + Apollo + Socket.io Server Entry
├── tailwind.config.ts      # Tailwind configuration (White/Slate + Indigo)
├── models/
│   ├── User.ts             # User Mongoose Schema
│   ├── Task.ts             # Task & Milestone Mongoose Schema
│   ├── Message.ts          # Real-time Chat Message Schema
```

```
│   └── Report.ts                 # Dispute & Report Schema
├── utils/
│   └── sendEmail.ts              # Brevo SMTP Nodemailer email dispatch utility
├── graphql/
│   ├── schema.ts                 # GraphQL Type Definitions (TypeDefs)
│   └── resolvers.ts              # Backend Query & Mutation Resolvers
└── app/                          # Next.js 14+ App Router
    ├── layout.tsx                # Root layout with providers & Tailwind styling
    ├── page.tsx                  # Detailed Public Landing Page
    ├── login/page.tsx            # Unified User Login Page
    ├── register/page.tsx         # Unified User Registration with Role Selector
    ├── jobs/
    │   ├── page.tsx              # Browse & Search Job Board (Fiverr: Gig List)
    │   └── [jobId]/page.tsx      # Job Details & Application View (Fiverr: Gig Detail)
    ├── dashboard/
    │   ├── employer/page.tsx     # Employer Control Center (Fiverr: Buyer Requests)
    │   ├── freelancer/page.tsx   # Freelancer Dashboard & Active Contracts
    │   └── admin/page.tsx        # Admin Moderation & User Management Panel
    └── workspace/
        └── [taskId]/page.tsx     # Active Contract Workspace (Chat + Escrow Actions)
```

4. Backend Implementation: Database Models (Mongoose)

A. User Model (models/User.ts)

Includes role differentiation (buyer vs. seller in Fiverr terms, though we use employer and freelancer).

TypeScript

```
import mongoose, { Schema, Document } from 'mongoose';

export interface IUser extends Document {
  name: string;
  email: string;
  passwordHash: string;
  role: 'employer' | 'freelancer' | 'admin';
  bio?: string;
  skills?: string[];
  paystackRecipientCode?: string; // For freelancer payouts
```

```
  createdAt: Date
}

const UserSchema = new Schema<IUser>({
  name: { type: String, required: true },
  email: { type: String, required: true, unique: true, lowercase: true },
  passwordHash: { type: String, required: true },
  role: { type: String, enum: ['employer', 'freelancer', 'admin'], default: 'freelancer' },
  bio: { type: String, default: '' },
  skills: [{ type: String }],
  paystackRecipientCode: { type: String },
}, { timestamps: true });

export default mongoose.models.User || mongoose.model<IUser>('User', UserSchema);
```

B. Task Model (models/Task.ts)

This is the core transaction object. An employer (buyer) posts a task. A freelancer (seller) applies.

```
TypeScript
import mongoose, { Schema, Document } from 'mongoose';

export interface ITask extends Document {
  employer: mongoose.Types.ObjectId;
  freelancer?: mongoose.Types.ObjectId;
  title: string;
  description: string;
  amount: number; // Stored in NGN
  category: 'Development' | 'Design' | 'Writing' | 'Marketing';
  status: 'pending_approval' | 'open' | 'funded' | 'in_progress' | 'submitted' | 'completed' | 'disputed';
  paystackReference?: string;
  createdAt: Date;
}

const TaskSchema = new Schema<ITask>({
  employer: { type: Schema.Types.ObjectId, ref: 'User', required: true },
  freelancer: { type: Schema.Types.ObjectId, ref: 'User' },
  title: { type: String, required: true },
  description: { type: String, required: true },
```

```
  amount: { type: Number, required: true },
  category: { type: String, enum: ['Development', 'Design', 'Writing', 'Marketing'], required: true },
  status: {
    type: String,
    enum: ['pending_approval', 'open', 'funded', 'in_progress', 'submitted', 'completed', 'disputed'],
    default: 'open'
  },
  paystackReference: { type: String },
}, { timestamps: true });

export default mongoose.models.Task || mongoose.model<ITask>('Task', TaskSchema);
```

C. Message Model (models/Message.ts)

For real-time chat inside the workspace.

```
TypeScript
import mongoose, { Schema, Document } from 'mongoose';

export interface IMessage extends Document {
  task: mongoose.Types.ObjectId;
  sender: mongoose.Types.ObjectId;
  content: string;
  createdAt: Date;
}

const MessageSchema = new Schema<IMessage>({
  task: { type: Schema.Types.ObjectId, ref: 'Task', required: true },
  sender: { type: Schema.Types.ObjectId, ref: 'User', required: true },
  content: { type: String, required: true },
}, { timestamps: true });

export default mongoose.models.Message || mongoose.model<IMessage>('Message',
MessageSchema);
```

D. Report Model (models/Report.ts)

For disputes handled by the admin.

```
TypeScript
import mongoose, { Schema, Document } from 'mongoose';

export interface IReport extends Document {
  task: mongoose.Types.ObjectId;
  reporter: mongoose.Types.ObjectId;
  reason: string;
  status: 'open' | 'resolved' | 'dismissed';
  createdAt: Date;
}

const ReportSchema = new Schema<IReport>({
  task: { type: Schema.Types.ObjectId, ref: 'User', required: true }, // Fixed schema error: should refer to Task, not User
  reporter: { type: Schema.Types.ObjectId, ref: 'User', required: true },
  reason: { type: String, required: true },
  status: { type: String, enum: ['open', 'resolved', 'dismissed'], default: 'open' }
}, { timestamps: true });

// Corrected Ref:
ReportSchema.path('task').ref('Task');

export default mongoose.models.Report || mongoose.model<IReport>('Report', ReportSchema);
```

5. Backend Utility: Brevo SMTP Email Service (utils/sendEmail.ts)

```
TypeScript
import nodemailer from 'nodemailer';

const transporter = nodemailer.createTransport({
  host: process.env.EMAIL_HOST, // smtp-relay.brevo.com
  port: Number(process.env.EMAIL_PORT) || 587,
  secure: false,
  auth: {
    user: process.env.EMAIL_USER,
```

```
    pass: process.env.EMAIL_PASS,
  },
});

export const sendEmail = async (to: string, subject: string, htmlContent: string) => {
  try {
    await transporter.sendMail({
      from: `"FreelanceX Notifications" <${process.env.EMAIL_FROM}>`,
      to,
      subject,
      html: htmlContent,
    });
    console.log(`Brevo email successfully dispatched to ${to}`);
  } catch (error) {
    console.error(`Brevo email dispatch error for ${to}:`, error);
  }
};
```

6. Backend Server Entry & Socket.io Real-Time Setup (server.js)

```
JavaScript
const express = require('express');
const http = require('http');
const { Server } = require('socket.io');
const mongoose = require('mongoose');
const cors = require('cors');
require('dotenv').config();

const Message = require('./models/Message');

const app = express();
const server = http.createServer(app);
const io = new Server(server, {
  cors: {
    origin: process.env.NEXT_PUBLIC_SITE_URL || 'http://localhost:3000',
    methods: ['GET', 'POST'],
  }
```

```
});

app.use(cors());
app.use(express.json());

// MongoDB Connection
mongoose.connect(process.env.MONGO_URI)
  .then(() => console.log('MongoDB Connected Successfully for FreelanceX'))
  .catch(err => console.error('MongoDB Connection Error:', err));

// Socket.io Real-Time Messaging Handler
io.on('connection', (socket) => {
  console.log(`User connected: ${socket.id}`);

  socket.on('join_task_room', (taskId) => {
    socket.join(taskId);
    console.log(`Socket ${socket.id} joined task room: ${taskId}`);
  });

  socket.on('send_message', async (data) => {
    try {
      const { taskId, senderId, content } = data;
      const newMessage = await Message.create({ task: taskId, sender: senderId, content });
      const populatedMessage = await Message.findById(newMessage._id).populate('sender', 'name email role');

      io.to(taskId).emit('receive_message', populatedMessage);
    } catch (error) {
      console.error('Socket message error:', error);
    }
  });

  socket.on('disconnect', () => {
    console.log(`User disconnected: ${socket.id}`);
  });
});

const PORT = process.env.PORT || 5000;
server.listen(PORT, () => {
  console.log(`FreelanceX Backend Server running on port ${PORT}`);
});
```

7. GraphQL API Specification (graphql/schema.ts)

Defines the contract for querying and mutating data.

```
GraphQL
type User {
  id: ID!
  name: String!
  email: String!
  role: String!
  bio: String
  skills: [String]
  createdAt: String!
}

type Task {
  id: ID!
  title: String!
  description: String!
  amount: Float!
  category: String!
  status: String!
  employer: User!
  freelancer: User
  paystackReference: String
  createdAt: String!
}

type Message {
  id: ID!
  task: ID!
  sender: User!
  content: String!
  createdAt: String!
}

type Report {
  id: ID!
  task: Task!
  reporter: User!
```

```
  reason: String
  status: String!
  createdAt: String!
}

type Query {
  getJobs: [Task!]!
  getJob(id: ID!): Task
  getMyTasks(userId: ID!, role: String!): [Task!]!
  getMessages(taskId: ID!): [Message!]!
  getUsers: [User!]! # Admin only
  getReports: [Report!]! # Admin only
}

type Mutation {
  registerUser(name: String!, email: String!, password: String!, role: String!, skills: [String!]): User!
  createTask(employerId: ID!, title: String!, description: String!, amount: Float!, category: String!): Task!
  updateTaskStatus(taskId: ID!, status: String!): Task!
  fileReport(taskId: ID!, reporterId: ID!, reason: String!): Report!
  deleteUser(userId: ID!): Boolean! # Admin only
}
```

8. Paystack Payment & Brevo Email Workflow

This section details the *Fiverr-like* secure transaction flow.

A. Initialize Payment Route (app/api/paystack/initialize/route.ts)

When an employer (buyer) clicks "Proceed to Pay", this route is called via Next.js. It initiates the transaction with Paystack, moving funds into escrow.

```
TypeScript
import { NextResponse } from 'next/server'
import axios from 'axios'
import Task from '@/models/Task'

export async function POST(req: Request) {
  try {
    const { taskId } = await req.json();
```

```
  const task = await Task.findById(taskId).populate('employer')

  if (!task) {
    return NextResponse.json({ error: 'Task not found' }, { status: 404 })
  }

  // Paystack requires amount in KOBO. (Naira * 100)
  const amountInKobo = task.amount * 100

  const response = await axios.post(
    'https://api.paystack.co/transaction/initialize',
    {
      email: task.employer.email,
      amount: amountInKobo,
      callback_url: `${process.env.NEXT_PUBLIC_SITE_URL}/workspace/${taskId}?verify=true`, // Redirect after payment
      metadata: { taskId: task._id.toString() }
    },
    {
      headers: {
        Authorization: `Bearer ${process.env.PAYSTACK_SECRET_KEY}`,
        'Content-Type': 'application/json'
      }
    }
  )

  return NextResponse.json({ authorization_url: response.data.data.authorization_url })
} catch (error: any) {
  return NextResponse.json({ error: error.response?.data?.message || error.message }, { status: 500 })
}
}
```

B. Automated Payout Transfer & Brevo Email Dispatch

When the employer clicks **"Approve & Release Funds"** (Fiverr: *Mark as Completed*), funds move from escrow to the freelancer's bank account.

```
TypeScript
import axios from 'axios'
```

```
import { sendEmail } from '@/utils/sendEmail'
import Task from '@/models/Task'
import User from '@/models/User'

export const releaseFundsToFreelancer = async (taskId: string) => {
  const task = await Task.findById(taskId).populate('freelancer employer')
  if (!task || task.status !== 'submitted') throw new Error('Invalid task state for payout.')

  const amountInKobo = task.amount * 100

  // 1. Trigger Paystack Transfer (Depositing funds into Freelancer's bank account)
  const transferResponse = await axios.post(
    'https://api.paystack.co/transfer',
    {
      source: "balance",
      amount: amountInKobo,
      recipient: task.freelancer.paystackRecipientCode, // Created when freelancer sets up bank details
      reason: `Payout for completed task: ${task.title}`
    },
    {
      headers: { Authorization: `Bearer ${process.env.PAYSTACK_SECRET_KEY}` }
    }
  )

  // 2. Update Task Status in Database
  task.status = 'completed'
  await task.save()

  // 3. Notify Freelancer via Brevo Email
  await sendEmail(
    task.freelancer.email,
    'Funds Released for Completed Task!',
    `<h2>Congratulations ${task.freelancer.name}!</h2><p>Your task <strong>${task.title}</strong> has been approved by ${task.employer.name}, and ₦${task.amount} has been successfully transferred to your bank account via Paystack.</p>`
  )

  // Notify Employer
  await sendEmail(
    task.employer.email,
    'Project Completed Successfully',
    `<h1>Project Closed</h1><p>The payment for ${task.title} has been released to
```

```
${task.freelancer.name}. Thank you for using FreelanceX.</p>`
  })

  return transferResponse.data;
}
```

9. Frontend & UI Pages Specification (Tailwind CSS)

All pages utilize the defined clean white and indigo palette:

- Backgrounds: bg-slate-50 (off-white), bg-white (cards/surfaces)
- Text: text-heading (#0F172A - charcoal), text-bodyText (#475569 - slate gray)
- Buttons: bg-primary (#4F46E5 - indigo), hover:bg-primary-hover

A. Authentication & Registration Flows

Unified Sign In / Login (app/login/page.tsx)

- *Fiverr UI Parallels:* Simple modal/page with tabs for Email/Username and Password.
- *Features:* Clean form, Zod validation for email/password. Upon successful login (via LOGIN_MUTATION), JWT token is stored, and user is redirected based on role (Employers to /dashboard/employer, Freelancers to /dashboard/freelancer, Admins to /dashboard/admin).

Unified Registration (app/register/page.tsx)

- *Fiverr UI Parallels:* Account creation screen.
- *Features:* Requires name, email, password. Includes a critical **Role Selector Radio Button**: "Are you looking to *Hire talent* (Employer) or *Find work* (Freelancer)?" This selection determines the role field in the User schema when REGISTER_MUTATION is called.

B. Discovery & Job Board Flows

Job Board / Browse Gigs (app/jobs/page.tsx)

- *Fiverr UI Parallels:* The central marketplace landing page, displaying available "Gigs" (Jobs).
- *Features:* Fetched via GET_JOBS_QUERY.
  - Search Bar (<input>) for filtering by title, description, or category.
  - Category Filters (<select>: Development, Design, Writing, Marketing).
  - Budget Filter (Slider/Input: Maximum Amount in NGN).
  - Grid of Task Cards (<TaskCard/>): Displays Title, Employer Name, Category Label, Budget (formatted as ₦15,000.00 using Intl.NumberFormat('en-NG')), and "Funded" metadata badge.
  - CTA button: "View Details" links to app/jobs/[jobId].

Job Details & Application (app/jobs/[jobId]/page.tsx)

- *Fiverr UI Parallels:* The Gig detail page.
- *Features:* Fetched via GET_JOB_QUERY using :jobId.
  - Displays Full Description, Budget, Category, and Employer profile.
  - *If User is Employer:* Button is disabled ("You cannot apply to your own job").

- *If User is Freelancer:* Button is visible: "Apply Now / Submit Proposal". Clicking this button calls the createTask mutation, assigning the freelancer to the task, changing the task status to pending_approval, and opening the workspace.

C. Employer (Buyer) Workflows

Employer Dashboard (app/dashboard/employer/page.tsx)

- *Fiverr UI Parallels:* Seller Dashboard / Buyer Request section.
- *Features:* Fetched via GET_MY_TASKS(userId, role: 'employer').
  - Summary Cards: Total Jobs Posted, Total Spend, Active Contracts.
  - Tabs: "All Jobs", "Active Projects", "Proposals Received".
  - List of Jobs Posted by this employer: Clicking a job shows a sub-list of freelancers who have applied.

Post New Job / Create Gig (app/dashboard/employer/create-job)

- *Fiverr UI Parallels:* Create a New Gig form.
- *Features:* A protected route accessible only to Role === 'employer'.
  - Standard Form (React Hook Form): Title (<input>), Description (<textarea>), Amount (<input> - validated in NGN), Category (<select>).
  - Submit Button calls CREATE_TASK_MUTATION. Success redirects to the Employer Dashboard.

D. Freelancer (Seller) Workflows

Freelancer Dashboard (app/dashboard/freelancer/page.tsx)

- *Fiverr UI Parallels:* Seller Dashboard.
- *Features:* Fetched via GET_MY_TASKS(userId, role: 'freelancer').
  - Summary Cards: Active Contracts, Total Earnings, Pending Proposals.
  - List of tasks where freelancerId === currentUserId. Task list items show the current state (e.g., "Funded", "Submitted", "Completed"). Clicking any item redirects to the shared app/workspace/[taskId].

E. Shared Transactional & Moderation Flows

Active Task Workspace (app/workspace/[taskId]/page.tsx)

- *Fiverr UI Parallels:* The main Order Page / Order Conversation.
- *Features:* A shared, protected route for both employer and freelancer involved in a specific task.
  - *Left Panel (Transaction Actions):*
    - If Status is open: Employer sees "Fund Task via Paystack" button. Freelancer sees "Awaiting Funding".
    - If Status is funded: Freelancer sees "Submit Work" button. Employer sees "Project Funded, Work in Progress".
    - If Status is submitted: Employer sees "Approve & Release Funds" button. Freelancer sees "Awaiting Approval".
    - If Status is disputed: Both see "Report Filed - Pending Admin Review".
    - "File Report / Dispute" button visible at all stages.
  - *Right Panel (Real-Time Chat):* Connected via Socket.io. <ChatRoom taskId="{taskId}"/>.

Loads previous messages from MongoDB (GET_MESSAGES_QUERY), and listens for new messages (receive_message).

Admin Dashboard (app/dashboard/admin/page.tsx)

- *Fiverr UI Parallels:* Internal Support/Trust & Safety Panel.
- *Features:* A protected route accessible only to Role === 'admin'.
  - Tabs: "All Users", "Open Reports (Disputes)".
  - User Management Table (via GET_USERS_QUERY): Lists all registered users (name, email, role) with a "Remove User" button (calls DELETE_USER_MUTATION).
  - Reports Table (via GET_REPORTS_QUERY): Lists active disputes. Clicking a report shows the related task details and allows the admin to communicate with both parties or resolve the escrow funds.

10. Acceptance Criteria for Development Completion

To consider the FreelanceX platform "shipped," the following criteria must be validated:

- [ ] **Authentication:** Users can register as Employer or Freelancer, sign in securely via JWT, and access role-restricted routes.
- [ ] **Job Board:** The marketplace is searchable and filterable, displaying live data from MongoDB.
- [ ] **Employer Actions:** Employers can post jobs, review applications, and initiate payments securely via Paystack.
- [ ] **Freelancer Actions:** Freelancers can apply for jobs and receive secure payouts directly to their bank accounts upon milestone approval.
- [ ] **Real-Time Chat:** Socket.io provides instant, persistent messaging within the Active Task Workspace.
- [ ] **Disputes & Admin:** Reports are filed correctly, and administrators can moderate users and resolve financial disputes.
- [ ] **Email Notifications:** Transactional emails (registration, task funding, payout) are successfully delivered via Brevo SMTP.
- [ ] **Design System:** The UI strictly adheres to the specified clean white, slate, and indigo visual identity.
- [ ] **TypeScript:** The codebase compiles without errors (tsc --noEmit).
- [ ] **Security:** No hardcoded credentials exist in the source code; all secret keys are loaded exclusively from the .env file.

Sources

1. https://github.com/BhargavK001/Hospital_Management_System_